



Instituto Politécnico Nacional

Escuela Superior de Cómputo



Práctica 1:

“Modelo de entidad relación”

Unidad de aprendizaje: Base de Datos

Profesor: Gabriel Hurtado Aviles

Carrera: Ingeniería en Sistemas Computacionales / 2020

Grupo: 3CV2

Integrantes:

- Aguilar García Ian Andrew
- Quintanar Sanchez Alondra
- Vega Treviño Kevin Edú
-

Periodo: 27/1

Objetivo

Que usted describa los sistemas de base de datos a partir de sus aspectos básicos, construya el entorno de trabajo que empleará durante todo el semestre -control de versiones y un sistema gestor de bases de datos ejecutado en un contenedor-, sustente su investigación con literatura científica arbitrada y aplique lo anterior al modelado conceptual de un problema real. Toda evidencia se entrega en un repositorio de Git.

Esta práctica cubre la Unidad Temática I completa y, al mismo tiempo, establece la forma de trabajo del resto del curso. Se divide en cinco ejercicios: los dos primeros construyen el entorno técnico, el tercero y el cuarto sustenta la parte conceptual, y el quinto la aplica a un caso concreto. La entrega no es un archivo suelto: es un repositorio de Git donde quedará registrado como trabajó usted, no solo el resultado final.

Ejercicio 1. Control de versiones con Git y GitHub

- **¿Qué es un sistema de control de versiones y qué problema concreto resuelve en un trabajo en equipo?**

En una sinfonía, varios instrumentos tocan sincronizadamente gracias al director. Pero ¿qué pasaría si quitáramos al director y los instrumentistas no pudieran escucharse unos a otros? La sincronización se perdería y solo quedaría un montón de ruido sin sentido. Este es un ejemplo exacto de cómo es desarrollar sin un sistema de control de versiones.

Los sistemas de control de versiones (VCS, por sus siglas en inglés) brindan a los equipos de programación una visibilidad completa del historial del código y una única fuente de referencia para todos los archivos, carpetas y comentarios. Estas herramientas optimizan el trabajo y minimizan la pérdida de tiempo al rastrear los cambios generados por el trabajo asíncrono. Cuando varias personas colaboran en un proyecto, es difícil mantener el seguimiento de las revisiones para saber quién cambió qué, cuándo y dónde están almacenados esos archivos.

Un sistema de control de versiones se ocupa de esos problemas manteniendo el registro de todas las modificaciones. Al igual que el historial en Microsoft Word o Google Drive, puedes tener un respaldo continuo de las versiones de tu código, de manera que el trabajo previo no se pierda en cada iteración. Esto hace que sea fácil regresar a una versión anterior y seguir contribuyendo al proyecto de forma segura.

- **¿Cuál es la diferencia entre Git y GitHub? No son sinónimos.**

Para poder saber las diferencias que hay entre Git y GitHub, primero aclaremos que es cada uno.

Git es un sistema de control de versiones distribuido gratis y de código abierto, siendo uno de los más utilizados. Fue creado por Linus Torvalds en 2005. Esta herramienta es un sistema de control de versiones que fue inicialmente desarrollado para trabajar con varios desarrolladores en un entorno de desarrollo en Linux.

El término “distribuido” significa que cuando le instruyes a Git que comparta el directorio de un proyecto, Git no sólo comparte la última versión del archivo. En cambio, distribuye cada versión que ha registrado para ese proyecto.

Esto hace que se diferencia de otros sistemas de control de versiones que solo comparten cualquier versión individual que un usuario haya extraído específicamente desde la base de datos central/local. Mientras que Git, por el contrario, no solo distribuyen algunas versiones previamente seleccionadas, sino todas las versiones de los archivos que se hayan registrado.

GitHub es una plataforma basada en la web donde los usuarios pueden alojar sus repositorios Git. Facilitar compartir y colaborar en proyectos con cualquier persona en cualquier momento. Esta también fomenta la participación más amplia en proyectos de Código Abierto, al proporcionar una manera segura de editar archivos en repositorios de otros usuarios.

Ahora con las definiciones claras, pasemos a las diferencias:

	Git	GitHub
Función principal	Es un sistema de control de versiones distribuido que registra las distintas versiones de un archivo (o conjunto de archivos). Le permite a los usuarios acceder, comparar, actualizar y distribuir cualquiera de las versiones registradas en cualquier momento.	Es una plataforma de alojamiento para albergar tus repositorios Git en la web. Esto permite a los usuarios mantener sus repositorios remotos privados o abiertos para esfuerzos colaborativos.
Plataforma de operación	Los usuarios instalan y ejecutan Git en sus equipos locales. Esto significa que la mayoría de las operaciones de Git se pueden lograr sin una conexión a internet.	Es un servicio basado en la web que opera solamente en línea. Eso significa que necesita de estar conectado para hacer cualquier cosa en GitHub.
Creadores	Linus Torvalds comenzó Git en abril del 2005.	Chris Wanstrath, P. J. Hyett, Tom Preston-Werner y Scott Chacon

		fundaron GitHub.com en febrero de 2008.
Mantenedores	En julio de 2005, Linus Torvalds entregó el mantenimiento de Git a Junio C. Hamano, quien ha sido el principal mantenedor desde entonces.	En octubre de 2018, Microsoft compró GitHub.
Competidores	Algunas alternativas populares para Git son: Mercurial, Team Foundation Version Control (TFVC), Perforce Helix Core, Apache Subversion y IBM Rational ClearCase.	Los competidores más cercanos a GitHub son GitLab, Bitbucket, SourceForge, Cloud Source, Repositories y AWS CodeCommit.

“Git y GitHub son dos entidades diferentes que te ayudan a administrar y alojar archivos. En otras palabras, Git sirve para controlar las versiones de los archivos, mientras que GitHub es una plataforma para alojar tus repositorios de Git.” (Castellanos, 2021)

- **Defina, con sus palabras y con un ejemplo: repositorio, confirmación (commit), rama (branch), fusión (merge), conflicto de fusión, Pull Request, archivo .gitignore y archivo README.**

Repositorio:

Un repositorio es un almacén digital donde se guarda un proyecto, permite visualizar un historial de cambios de este. A su vez que sirve para organizar, gestionar y proteger archivos, código, documentación o reportes.

Un ejemplo de repositorio sería el Repositorio Institucional de la UNAM

Confirmación (commit):

Un commit es como un punto de guardado; sirve para registrar la versión con los nuevos cambios que hiciste en el proyecto.

Por ejemplo, acabas de hacer una actualización o arreglar un error, haces un commit con la descripción de lo que hiciste para guardar el avance o modificación en el historial.

Rama (Branch):

Es una línea temporal e independiente dentro del repositorio, que normalmente se usa para poder probar ideas y experimentar libremente sin afectar al código principal.

Sirve o se utiliza para poder probar funciones nuevas que se desean añadir, como probar agregar un “modo oscuro” en una página web o aplicación.

Fusión (Merge):

Es el unir a dos ramas en una sola, sirve para integrar la rama temporal o de prueba, a la rama principal.

Siguiendo con el ejemplo anterior, cuando ya terminaste de experimentar con tu rama de “modo oscuro” y estas listo para agregarla al código principal, haces un merge para que ese código se una a la rama principal.

Conflicto de fusión:

Cuando Git no puede escoger cual será la versión final o no puede fusionar las dos ramas que el usuario desea. Esto puede pasar por varias razones, como que más de un usuario a hecho cambios o ramas en la misma línea, cuando alguien quiere guardar un cambio en un archivo que alguien más borro, entre otros.

Por ejemplo, mientras yo hice una rama para probar mi función de “modo oscuro”, mi compañero de equipo por accidente lo borró, haciendo que cuando intente hacer la fusión se cree el conflicto de fusión.

Pull Request:

Es para hacer una solicitud o una sugerencia de fusión de ramas al equipo en el que estás trabajando, para que lo puedan analizar y discutir si autorizarlo o no. Sirve para evitar errores en el código principal y como herramienta para fortalecer la comunicación en el equipo.

Podría hacer una Pull Request para enseñarle a mi equipo la nueva función que quiero agregar al código de modo oscuro. Para que a mi equipo les llegue la notificación y me puedan dejar comentarios como “Te faltó cambiar el color de este botón”, antes de que se agregue a la línea principal, y en general llegar a un acuerdo unánime sobre si fusionar la rama al main o no.

Archivo .gitignore:

Es un archivo de texto que le indica a Git que archivos o carpetas debe ignorar.

Si por alguna razón tuviera en un archivo de texto mi contraseña de roblox porque se me olvida mucho, y por obvias razones no quisiera que mi equipo lo viera, escribo el nombre del archivo dentro del comentario .gitignore para que, git lo ignore y no lo suba al repositorio.

Archivo README:

Es un texto que te da la bienvenida por así decirlo al proyecto que estas visitando, ya que es lo primero que vas a ver al entrar al proyecto. Normalmente se usa para justificar o explicar el propósito de ese proyecto, aunque en otros ámbitos (como los mods en videojuegos) se utiliza para poner un pequeño tutorial de descarga, instalación y configuración.

Por ejemplo, quiero descargar un mod que cambia las texturas del menú en un videojuego, pero no sé mucho de programación o instalación de mods, pero, gracias que al entrar hay un

archivo README con las instrucciones de como debo instalarlo o configurarlo, en qué carpeta del juego debo guardar el mod, etc. Ahora puedo jugar felizmente con mi configuración en rosita.

- **En qué consiste un flujo de trabajo basado en ramas y por qué el código se revisa entre pares antes de fusionarse.**

Es un proceso que utilizan los programadores al trabajar con proyectos con varias ramas. Se mantiene una rama principal (o main) que es el código base, y si quieren crear o probar nuevas funciones, hacen otras ramas, para finalmente, fusionarlas con la rama principal o main.

El código se revisa entre pares antes de fusionar la rama al main, para que, otro compañero distinto al que escribió el código, pueda revisarlo y analizarlo con detenimiento. Esto ayuda a reducir los errores, encontrarles soluciones o hasta retroalimentación desde ambas partes. De esta manera, se fortalece y mejorando el trabajo en equipo, la productividad y los resultados.

Bibliografía

Acerca de los pull requests - Documentación de GitHub. (s. f.). GitHub Docs.

<https://docs.github.com/es/pull-requests/get-started/about-pull-requests>

Acerca del archivo README del repositorio - Documentación de GitHub. (s. f.). GitHub Docs.

<https://docs.github.com/es/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>

Atlassian. (2025, diciembre 15). *Archivo .gitignore: ignorar archivos en Git.* Atlassian.

<https://www.atlassian.com/es/git/tutorials/saving-changes/gitignore>

Atlassian. (s. f.). *Flujo de trabajo de Gitflow.* Atlassian.com. Retrieved September 16, 2026, from

<https://www.atlassian.com/es/git/tutorials/comparing-workflows/gitflow-workflow>

Amazon Web Services. (s. f.). *¿Qué es Repo?* Amazon Web Services, Inc.

<https://aws.amazon.com/es/what-is/repo/>

Castellanos, E. (2021, 14 febrero). *Git vs GitHub – ¿Qué es el Control de Versiones y Cómo Funciona?*

freeCodeCamp.org. <https://www.freecodecamp.org/espanol/news/git-vs-github-what-is-version-control-and-how-does-it-work/>

Conflictos de fusión - Documentación de GitHub. (s. f.). GitHub Docs. <https://docs.github.com/es/pull-requests/reference/merge-conflicts>

Flujos de trabajo - Documentación de GitHub. (s. f.). GitHub Docs.

<https://docs.github.com/es/actions/concepts/workflows-and-actions/workflows>

Git - Branching workflows. (s. f.). <https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows>

GitLab. (s. f.). *¿Qué es una revisión de código?* GitLab. <https://about.gitlab.com/es/topics/version-control/what-is-code-review/#four-approaches-to-code-review>

Git - Procedimientos Básicos para Ramificar y Fusionar. (s. f.). https://git-scm.com/book/es/v2/Ramificaciones-en-Git-Procedimientos-B%C3%A1sicos-para-Ramificar-y-Fusionar#r_basic_merge_conflicts

De Repositorios Universitarios Universidad Nacional Autónoma de México, D. G. (2021, 7 mayo).

Repositorio Institucional de la UNAM. <https://repositorio.unam.mx/>

Git - ramificar y fusionar. (s. f.). <https://git-scm.com/book/es/v2/Ap%C3%A9ndice-C:-Comandos-de-Git-Ramificar-y-Fusionar>

Kern-It. (2026, 9 septiembre). *Merge.* <https://www.kern-it.be/en/definitions/merge/>

Make a README. (s. f.). Make A README. <https://www.makeareadme.com/>

Montiel, O. (2021, enero 31). *Explicación de Git branch: Cómo eliminar, mover, crear y renombrar una rama en Git.* freecodecamp.org. <https://www.freecodecamp.org/espanol/news/explicacion-de-la-rama-de-gi-como-eliminar/>

MSMK. (2025, 19 marzo). *Branch en Programación*. <https://msmk.university/branch-en-programacion/>

Sulbaran, C. (2021, 19 enero). *Gitignore Explicado: ¿Qué es Gitignore, y cómo agregarlo a tu repositorio?* freeCodeCamp.org. <https://www.freecodecamp.org/espanol/news/gitignore-explicado-que-es-y-como-agregar-a-tu-repositorio/>

What is Commit? What does it do? (s. f.-b). Techcareer.net | Artificial Intelligence (AI) Competency Academy. <https://www.techcareer.net/en/dictionary/commit>